

CATALOG API PROJESİ

A'dan Z'ye Satır Satır Kod Açıklamaları ve Mimari El Kitabı

Teknolojiler: .NET 10 | Clean Architecture | EF Core | PostgreSQL | JWT & Identity | Docker

Compose

Geliştirici: Duru Rençber

Tarih: Ekim 2026

Bölüm 1: Kimlik Doğrulama ve DTO Modelleri

Dosya: CatalogAPI.API/DTOs/AuthDtos.cs

API Katmanı / Veri Transfer Nesneleri

```
namespace CatalogAPI.API.DTOs;  
  
public record RegisterDto(string Email, string Password, string Role);  
public record LoginDto(string Email, string Password);  
public record AuthResponseDto(string Token, string Email, string Role);
```

Satır Satır Açıklama:

- `namespace CatalogAPI.API.DTOs;` : Bu sınıfların proje içindeki mantıksal adresini belirtir (C# File-scoped namespace).

- `public record RegisterDto(...)` : Kayıt olacak kullanıcının dışarıdan göndermek zorunda olduğu alanları temsil eder. C#'taki `record` anahtar kelimesi, değişmez (immutable) ve hafif veri taşıyıcıları üretir.
- `public record LoginDto(...)` : Kullanıcı sisteme girerken yalnızca e-posta ve şifre ister; rol veya ID gibi alanlar kullanıcı tarafından belirlenemez.
- `public record AuthResponseDto(...)` : Başarılı giriş sonrası istemciye (tarayıcıya veya curl'e) geri döneceğimiz veri paketidir. Üretilen JWT token dizesi, kullanıcının e-postası ve yetki seviyesi (rolü) burada yer alır.

Bölüm 2: Kimlik Doğrulama Kontrolcüsü (AuthController)

Dosya: CatalogAPI.API/Controllers/AuthController.cs

API Katmanı / Giriş Kapısı

```
[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly UserManager<IdentityUser> _userManager;
    private readonly RoleManager<IdentityRole> _roleManager;
    private readonly IConfiguration _configuration;

    public AuthController(
        UserManager<IdentityUser> userManager,
        RoleManager<IdentityRole> roleManager,
        IConfiguration configuration)
    {
        _userManager = userManager;
        _roleManager = roleManager;
        _configuration = configuration;
    }
}
```

Satır Satır Açıklama:

- `[ApiController]` : ASP.NET Core'a bu sınıfın bir Web API olduğunu bildirir. Model doğrulama (validation) hatalarını otomatik olarak 400 Bad Request şeklinde döner.
- `[Route("api/[controller]")]` : URL rotasını dinamik belirler. Controller adı `Auth` olduğu için istekler `/api/auth` adresine yönlendirilir.
- `UuserManager<IdentityUser>` : Microsoft Identity kütüphanesinin kullanıcı oluşturma, şifre hash'leme ve kullanıcı sorgulama motorudur.
- `RoleManager<IdentityRole>` : Sistemdeki rolleri (Admin, User) yöneten servistir.
- `IConfiguration` : `appsettings.json` veya ortam değişkenlerindeki JWT gizli anahtarlarını okumak için enjekte edilir (Dependency Injection).

Kayıt Olma Metodu: Register

```
[HttpPost("register")]
public async Task<IActionResult> Register([FromBody] RegisterDto dto)
```

```

{
    var userExists = await _userManager.FindByEmailAsync(dto.Email);
    if (userExists != null)
        return BadRequest("Bu e-posta adresiyle kayıtlı kullanıcı zaten var.");

    var user = new IdentityUser { UserName = dto.Email, Email = dto.Email };
    var result = await _userManager.CreateAsync(user, dto.Password);

    if (!result.Succeeded)
        return BadRequest(result.Errors);

    if (!await _roleManager.RoleExistsAsync(dto.Role))
        await _roleManager.CreateAsync(new IdentityRole(dto.Role));

    await _userManager.AddToRoleAsync(user, dto.Role);

    return Ok(new { message = "Kullanıcı başarıyla kaydedildi." });
}

```

Satır Satır Açıklama:

- `[HttpPost("register")]` : `/api/auth/register` adresine gelen POST isteklerini karşılar.
- `await _userManager.FindByEmailAsync(dto.Email);` : Aynı e-posta ile ikinci bir hesap açılmasını engellemek için veritabanında arama yapar.
- `var result = await _userManager.CreateAsync(user, dto.Password);` : Şifreyi asla düz metin (plain text) olarak kaydetmez; PBKDF2 algoritmasıyla tuzlayarak (salt) güvenli hash üretir ve veritabanına yazar.
- `if (!await _roleManager.RoleExistsAsync(dto.Role))` : Kullanıcının talep ettiği rol ("Admin") sistemde henüz tanımlı değilse dinamik olarak oluşturur.
- `await _userManager.AddToRoleAsync(user, dto.Role);` : Kullanıcıyı veritabanında rolüyle ilişkilendirir (`AspNetUserRoles` tablosuna yazar).

Giriş Yapma ve Token Üretme Metodu: Login

```

[HttpPost("login")]
public async Task<IActionResult> Login([FromBody] LoginDto dto)
{
    var user = await _userManager.FindByEmailAsync(dto.Email);
    if (user == null || !await _userManager.CheckPasswordAsync(user,
dto.Password))
        return Unauthorized("E-posta veya şifre hatalı.");

    var userRoles = await _userManager.GetRolesAsync(user);
    var role = userRoles.FirstOrDefault() ?? "User";

    var authClaims = new List<Claim>
    {

```

```
        new Claim(ClaimTypes.NameIdentifier, user.Id),
        new Claim(ClaimTypes.Email, user.Email!),
        new Claim(ClaimTypes.Role, role),
        new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
    };

    var authSigningKey = new SymmetricSecurityKey(
        Encoding.UTF8.GetBytes(_configuration["JwtSettings:SecretKey"]!));

    var token = new JwtSecurityToken(
        issuer: _configuration["JwtSettings:Issuer"],
        audience: _configuration["JwtSettings:Audience"],
        expires: DateTime.UtcNow.AddMinutes(
            Convert.ToDouble(_configuration["JwtSettings:ExpirationInMinutes"])),
        claims: authClaims,
        signingCredentials: new SigningCredentials(authSigningKey,
            SecurityAlgorithms.HmacSha256)
    );

    return Ok(new AuthResponseDto(new
        JwtSecurityTokenHandler().WriteToken(token), user.Email!, role));
}
```

Satır Satır Açıklama:

- `CheckPasswordAsync(...)` : Girilen şifreyi veritabanındaki hash ile matematiksel olarak kıyaslar. Uyuşmazsa 401 Unauthorized döner.
- `authClaims` : Token'ın içine gömülecek kimlik bilgileridir (Payload). Kullanıcı ID'si, e-postası ve Rolü burada mühürlenir.
- `JwtRegisteredClaimNames.Jti` : Her token'a benzersiz bir rastgele ID atar (Token replay saldırılarını önlemek için).
- `SymmetricSecurityKey` : `appsettings.json` içindeki gizli metni baytlara çevirerek simetrik şifreleme anahtarı yapar.
- `SigningCredentials(..., HmacSha256)` : Token'ın üçüncü parçası olan dijital imzayı üretir. İmzayı yalnızca bu sunucu doğrulayabilir.

Bölüm 3: Rol Korumalı Ürün Uç Noktaları

Dosya: CatalogAPI.API/Controllers/ProductsController.cs

API Katmanı / Güvenlik Duvarı

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    private readonly CatalogDbContext _context;

    public ProductsController(CatalogDbContext context)
    {
        _context = context;
    }

    [HttpGet]
    public async Task<IActionResult> GetAll()
    {
        var products = await _context.Products.ToListAsync();
        return Ok(products);
    }

    [HttpPost]
    [Authorize(Roles = "Admin")]
    public async Task<IActionResult> Create([FromBody] CreateProductDto dto)
    {
        var category = await _context.Categories.FindAsync(dto.CategoryId);
        if (category == null)
            return BadRequest("Belirtilen kategori bulunamadı.");

        var product = new Product
        {
            Id = Guid.NewGuid(),
            Name = dto.Name,
            Description = dto.Description,
            Price = dto.Price,
            StockQuantity = dto.Stock,
            CategoryId = dto.CategoryId
        };

        _context.Products.Add(product);
        await _context.SaveChangesAsync();

        return CreatedAtAction(nameof(GetAll), new { id = product.Id }, product);
    }
}
```

```
}  
}
```

Satır Satır Açıklama:

- `[HttpGet] GetAll()` : Üzerinde `[Authorize]` olmadığı için halka açıktır (public). Vitrindeki ürünleri herkes görebilir.
- `[Authorize(Roles = "Admin")]` : Bu metodun kapısına güvenlik görevlisi koyar. Gelen istekte geçerli bir JWT yoksa `401 Unauthorized` ; token var ama içinde "Admin" rolü yoksa `403 Forbidden` hatası fırlatır.
- `var category = await _context.Categories.FindAsync(dto.CategoryId);` : İlişkisel bütünlüğü (Referential Integrity) garanti eder. Olmayan bir kategori ID'si ile ürün açılmasını engeller (Yaşadığımız 400 hatasının kaynağı).
- `await _context.SaveChangesAsync();` : EF Core nesne takipçisindeki değişiklikleri PostgreSQL'e `INSERT INTO "Products"...` SQL sorgusu olarak gönderir.
- `CreatedAtAction(...)` : HTTP 201 Created durum koduyla birlikte `Location` header'ına yeni ürünün adresini ekler.

Bölüm 4: Altyapı ve Veritabanı Yapılandırması

Dosya: CatalogAPI.Infrastructure/Persistence/CatalogDbContext.cs

Infrastructure Katmanı / EF Core

```
namespace CatalogAPI.Infrastructure.Persistence;

public class CatalogDbContext : IdentityDbContext<IdentityUser>
{
    public CatalogDbContext(DbContextOptions<CatalogDbContext> options) :
base(options)
    {
    }

    public DbSet<Product> Products => Set<Product>();
    public DbSet<Category> Categories => Set<Category>();

    protected override void OnModelCreating(ModelBuilder builder)
    {
        base.OnModelCreating(builder);

        builder.Entity<Product>(entity =>
        {
            entity.HasKey(p => p.Id);
            entity.Property(p => p.Name).IsRequired().HasMaxLength(150);
            entity.Property(p => p.Price).HasPrecision(18, 2);
            entity.HasOne(p => p.Category)
                .WithMany(c => c.Products)
                .HasForeignKey(p => p.CategoryId);
        });
    }
}
```

Satır Satır Açıklama:

- `IdentityDbContext<IdentityUser>` : Standart DbContext yerine IdentityDbContext'ten miras aldık. Bu sayede veritabanına otomatik olarak ``AspNetUsers``, ``AspNetRoles``, ``AspNetUserClaims`` gibi 7 adet güvenlik tablosu eklendi.
- `base.OnModelCreating(builder);` : Identity tablolarının birincil anahtar (Primary Key) eşlemelerinin doğru çalışması için bu satırın ilk sırada çağırılması zorunludur.
- `entity.Property(p => p.Price).HasPrecision(18, 2);` : Para birimlerinde kuruş hassasiyetini korumak için PostgreSQL tarafında ``decimal(18,2)`` veri tipini zorunlu kılar.

- `entity.HasOne(...).WithMany(...)` : 1-N (Bire-Çok) ilişki kurar: Bir ürünün tek bir kategorisi olabilir, bir kategoride birden çok ürün bulunabilir.

Bölüm 5: Bağımlılık Enjeksiyonu ve Middleware Hattı

Dosya: CatalogAPI.API/Program.cs

API Katmanı / Uygulama Omurgası

```
var builder = WebApplication.CreateBuilder(args);

// 1. PostgreSQL Veritabanı Bağlantısı
builder.Services.AddDbContext<CatalogDbContext>(options =>

options.UseNpgsql(builder.Configuration.GetConnectionString("DefaultConnection")))
;

// 2. Identity Kurulumu
builder.Services.AddIdentity<IdentityUser, IdentityRole>()
    .AddEntityFrameworkStores<CatalogDbContext>()
    .AddDefaultTokenProviders();

// 3. JWT Kimlik Doğrulama Şeması
builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = builder.Configuration["JwtSettings:Issuer"],
        ValidAudience = builder.Configuration["JwtSettings:Audience"],
        IssuerSigningKey = new SymmetricSecurityKey(

Encoding.UTF8.GetBytes(builder.Configuration["JwtSettings:SecretKey"]!))
    };
});

var app = builder.Build();

app.UseAuthentication(); // 1. Sen kimsin? (Token çözülür)
app.UseAuthorization(); // 2. Buraya girmeye yetkin var mı? (Rol kontrol edilir)
```

```
app.MapControllers();  
app.Run();
```

Satır Satır Açıklama:

- `builder.Services.AddDbContext...` : Veritabanı bağlantı havuzunu (connection pool) oluşturur ve Controller'ların constructor'larında DbContext talep edebilmesini sağlar.
- `AddEntityFrameworkStores<CatalogDbContext>()` : Identity motorunun kullanıcı verilerini dosya veya belleğe değil, bizim PostgreSQL tablolarımıza yazmasını sağlar.
- `options.TokenValidationParameters` : Gelen her istekte token'ın süresinin dolup dolmadığını ('ValidateLifetime'), sunucu imzasının doğruluğunu ('ValidateIssuerSigningKey') otomatik test eder.
- `app.UseAuthentication()` ve `app.UseAuthorization()` : Sıralama hayatidir! Önce kimlik doğrulanmalı ('Authentication'), ardından yetki sorgulanmalıdır ('Authorization'). Sıralama ters olursa yetkilendirme her zaman başarısız olur.

Bölüm 6: Konteynerleştirme ve Orkestrasyon

Dosya: CatalogAPI.API/Dockerfile

DevOps / Multi-Stage Build

```
# 1. Aşama: Build & Publish (SDK İmajı - ~1 GB)
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /app

# Katman csproj dosyalarını kopyala ve restore et (Docker Cache Optimizasyonu)
COPY CatalogAPI.Domain/*.csproj CatalogAPI.Domain/
COPY CatalogAPI.Application/*.csproj CatalogAPI.Application/
COPY CatalogAPI.Infrastructure/*.csproj CatalogAPI.Infrastructure/
COPY CatalogAPI.API/*.csproj CatalogAPI.API/

RUN dotnet restore CatalogAPI.API/CatalogAPI.API.csproj

# Kalan tüm kodları kopyala ve derle
COPY . .
WORKDIR /app/CatalogAPI.API
RUN dotnet publish -c Release -o /app/out

# 2. Aşama: Runtime (Hafif ASP.NET İmajı - ~150 MB)
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
WORKDIR /app
COPY --from=build /app/out .

EXPOSE 8080
ENTRYPOINT ["dotnet", "CatalogAPI.API.dll"]
```

Satır Satır Açıklama:

- `FROM ... AS build` : Derleme için ağır olan SDK imajını geçici bir aşama olarak tanımlar.
- `COPY .../*.csproj ...` ve `RUN dotnet restore` : Kodlar değişse bile NuGet paketleri değişmediği sürece Docker bu adımı önbellekten (cache) alır. Böylece her derlemede internetten kütüphane indirilmez.
- `RUN dotnet publish -c Release -o /app/out` : Kodları optimize edilmiş, üretime hazır binary DLL'ler haline getirir ve `/app/out` dizinine çıkarır.
- `COPY --from=build /app/out .` : Ağır SDK aşamasındaki kod ve derleme çöplerini çöpe atar; sadece derlenmiş DLL'leri hafif Runtime imajına taşır. İmaj boyutu %80 küçülür.

Dosya: docker-compose.yml

DevOps / Servis Orkestrasyonu

```
services:
  db:
    image: postgres:16-alpine
    container_name: catalog_postgres
    restart: always
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: CatalogDb
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  catalog-api:
    build:
      context: .
      dockerfile: CatalogAPI.API/Dockerfile
    container_name: catalog_api
    restart: always
    depends_on:
      - db
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      - ConnectionStrings__DefaultConnection=Host=db;Port=5432;Database=CatalogDb;Username=postgres;Password=postgres
      - JwtSettings__Issuer=CatalogAPI
      - JwtSettings__Audience=CatalogAppUsers
      - JwtSettings__SecretKey=BuCokGizliVeGucluBirAnahtardirEnAz32KarakterOlmali!
      - JwtSettings__ExpirationInMinutes=60
    ports:
      - "5054:8080"

  catalog-web:
    build:
      context: .
      dockerfile: CatalogAPI.Web/Dockerfile
    container_name: catalog_web
    restart: always
    depends_on:
      - catalog-api
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      - ApiSettings__BaseUrl=http://catalog-api:8080/
    ports:
```

– "5001:8080"

volumes:
 postgres_data:

Satır Satır Açıklama:

- `services: db` : PostgreSQL veritabanını izole container olarak ayağa kaldırır. Alpine tabanlı hafif imaj kullanır.
- `volumes: postgres_data` : Container silinse veya yeniden başlatılsa dahi verilerin (kullanıcılar, ürünler) kaybolmaması için Docker üzerinde kalıcı disk alanı sağlar.
- `depends_on: - db` : `catalog-api` başlamadan önce mutlaka `db` servisinin ayağa kalkmasını garanti eder.
- `Host=db` : API, veritabanına bağlanırken `localhost` yerine Docker iç ağındaki container adı olan `db` ismini kullanır. Docker bu adı arka planda otomatik IP adresine çözümler.
- `ports: - "5054:8080"` : Host makinenin (Mac) 5054 portunu, container içindeki Kestrel web sunucusunun dinlediği 8080 portuna yönlendirir.
- `ApiSettings__BaseUrl=http://catalog-api:8080/` : Web (MVC) katmanının API ile doğrudan Docker iç ağı üzerinden yüksek hızda konuşmasını sağlar.